

Aula 17 - Integração Frontend-Backend em Java

Duração: 2 horas

Objetivos:

- Integrar frontend (Thymeleaf) com backend Spring Boot
- Consumir API REST com Java HTTP Client
- Implementar tratamento de erros frontend

Parte Teórica (30 min)

1. Modelos de Integração

Abordagem	Tecnologias	Casos de Uso
Server-side	Thymeleaf + Controller	Apps simples, SEO-friendly
Client-side	REST API + JavaScript	SPAs complexas
Híbrida	Ambas	Migrações progressivas

2. Fluxo de Comunicação

Diagram

Code

```
sequenceDiagram
    Browser->>+Controller: GET /produtos (Thymeleaf)
    Controller->>+Service: Chamada Java
    Service->>+Repository: Consulta DB
    Repository-->>-Service: List<Produto>
    Service-->>-Controller: ModelAndView
    Controller-->>-Browser: HTML + Data
```

Atividade Prática (1h30min)

Cenário: Listagem de Produtos

Passo 1: Controller Thymeleaf

java

```
// src/main/java/com/example/projeto/web/ProdutoController.java
@Controller
@RequestMapping("/produtos")
@RequiredArgsConstructor
public class ProdutoController {
    private final ProdutoService service;

    @GetMapping
    public String listar(Model model) {
        model.addAttribute("produtos", service.listarTodos());
        return "produtos/lista";
    }
}
```

Passo 2: Template Thymeleaf

html

```
<!-- src/main/resources/templates/produtos/lista.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Lista de Produtos</title>
</head>
<body>
    <h1>Produtos</h1>
    <table>
        <tr th:each="produto : ${produtos}">
            <td th:text="${produto.nome}"></td>
            <td th:text="${#numbers.formatCurrency(produto.preco)}"></td>
        </tr>
    </table>
</body>
</html>
```

Consumindo API Externa

Passo 3: HTTP Client

java

```
// src/main/java/com/example/projeto/integration/CepService.java
@Service
public class CepService {
    private final RestTemplate restTemplate;
```

```

    public Endereco buscarPorCep(String cep) {
        String url = "https://viacep.com.br/ws/" + cep + "/json/";
        return restTemplate.getForObject(url, Endereco.class);
    }
}

```

Passo 4: Tratamento de Erros

java

```

@ControllerAdvice
public class WebExceptionHandler {
    @ExceptionHandler({RestClientException.class})
    public String handleApiError(Model model) {
        model.addAttribute("error", "Falha ao consultar API externa");
        return "error";
    }
}

```

Atividade: Formulário de Cadastro

1. Controller

java

```

@PostMapping("/cadastrar")
public String cadastrar(@Valid ProdutoForm form, BindingResult result) {
    if (result.hasErrors()) {
        return "produtos/form";
    }
    service.cadastrar(form.toEntity());
    return "redirect:/produtos";
}

```

2. Template com Validação

html

```

<form th:action="@{/produtos/cadastrar}" th:object="${form}" method="post">
    <input th:field="*{nome}" type="text">
    <p th:if="${#fields.hasErrors('nome')}" th:errors="*{nome}"></p>

    <input th:field="*{preco}" type="number" step="0.01">

```

```
<button type="submit">Salvar</button>
</form>
```

Checklist de Entrega

- Página Thymeleaf funcional
- Integração com API externa
- Tratamento de erros básico
- PR com label `frontend-integration`

Tarefa Pós-Aula

1. Adicione um dropdown de categorias (consumindo outra API)
2. Implemente um loading animation durante chamadas AJAX

Exemplo AJAX:

javascript

```
// Em lista.html
fetch('/api/produtos')
  .then(response => response.json())
  .then(data => {
    // Atualiza tabela dinamicamente
  });
```

Próximos Passos

- Aula 18: Autenticação com Spring Security
- Aula 19: Deploy no Heroku

Dica Pro:

"Use `@Async` para chamadas API demoradas e evite bloquear threads!"

Recursos:

- [Thymeleaf Docs](#)
- [RestTemplate Guide](#)

Pronto para unir frontend e backend?  